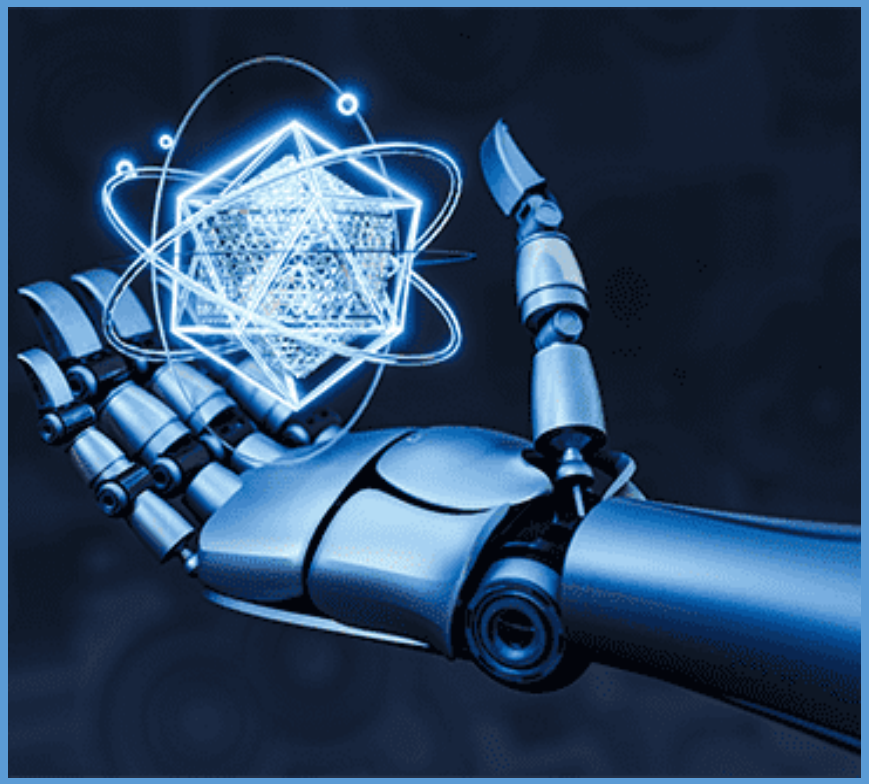
Applied AI & Machine Learning
CS-333

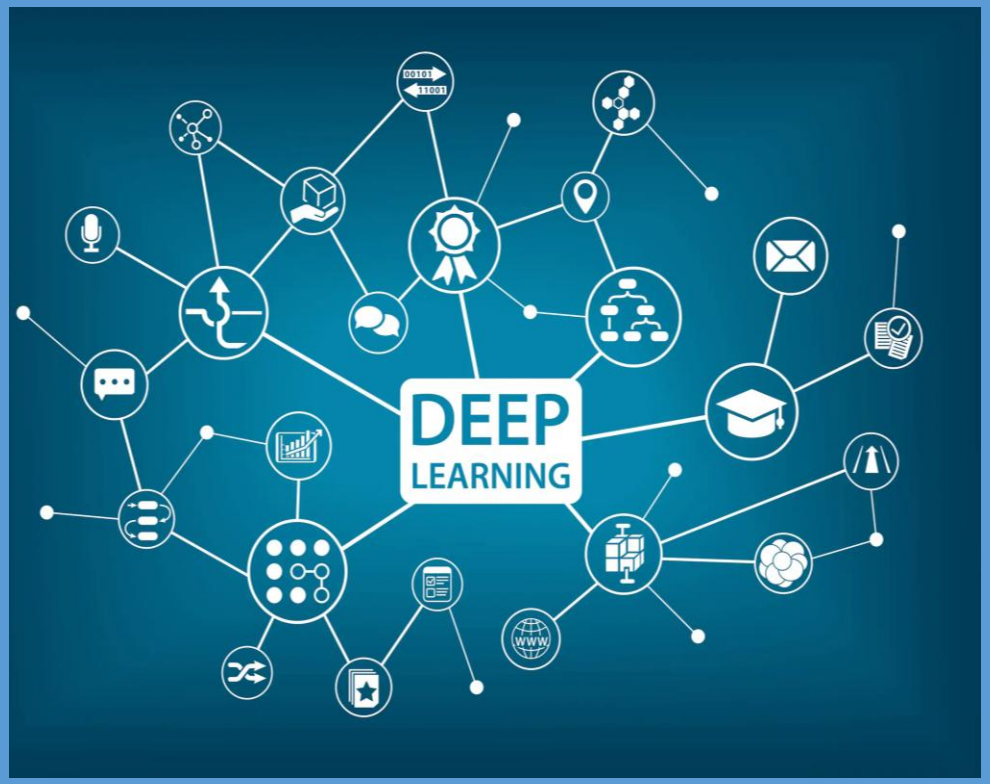
Dr. Abbas Hussain

PNEC, NUST

Lecture 9

Spring 2026





Introduction to Deep Learning

Learning Outcomes

By the end of this lecture, students will be able to:

- Understand the concept of adaptive learning in neural networks
- Explain batching and mini-batch training in deep learning
- Differentiate between epoch, batch size, and iteration
- Analyze the role of optimization in training neural networks
- Identify the problem of overfitting in machine learning models
- Distinguish between high bias and high variance
- Apply regularization techniques to improve model generalization
- Explain the working principle of dropout as a regularization method
- Evaluate the impact of batch size on training efficiency and model performance
- Describe how epochs, batches, and iterations interact during training

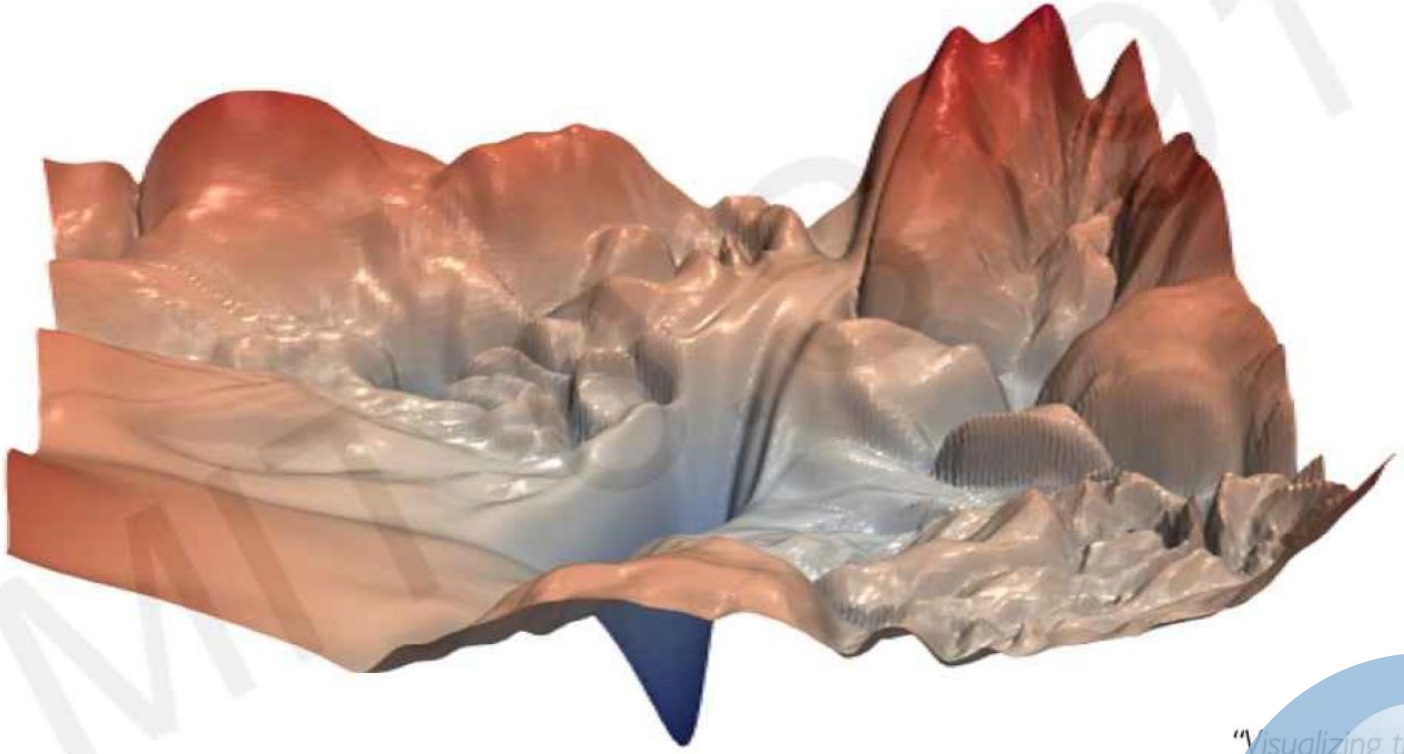
Topics

1. Adaptive Learning
2. Batching
3. Regularization
4. Epoch vs Batch vs Iteration

Lecture slides are taken from MIT 6.S191 Intro to deep learning course

Neural Networks in Practice: Optimization

Training Neural Networks is Difficult



"Visualizing the loss landscape of neural nets". Dec 2017.

Loss Functions Can Be Difficult to Optimize

Remember:

Optimization through gradient descent

$$\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta \frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}}$$

MIT 6.S191

Loss Functions Can Be Difficult to Optimize

Remember:

Optimization through gradient descent

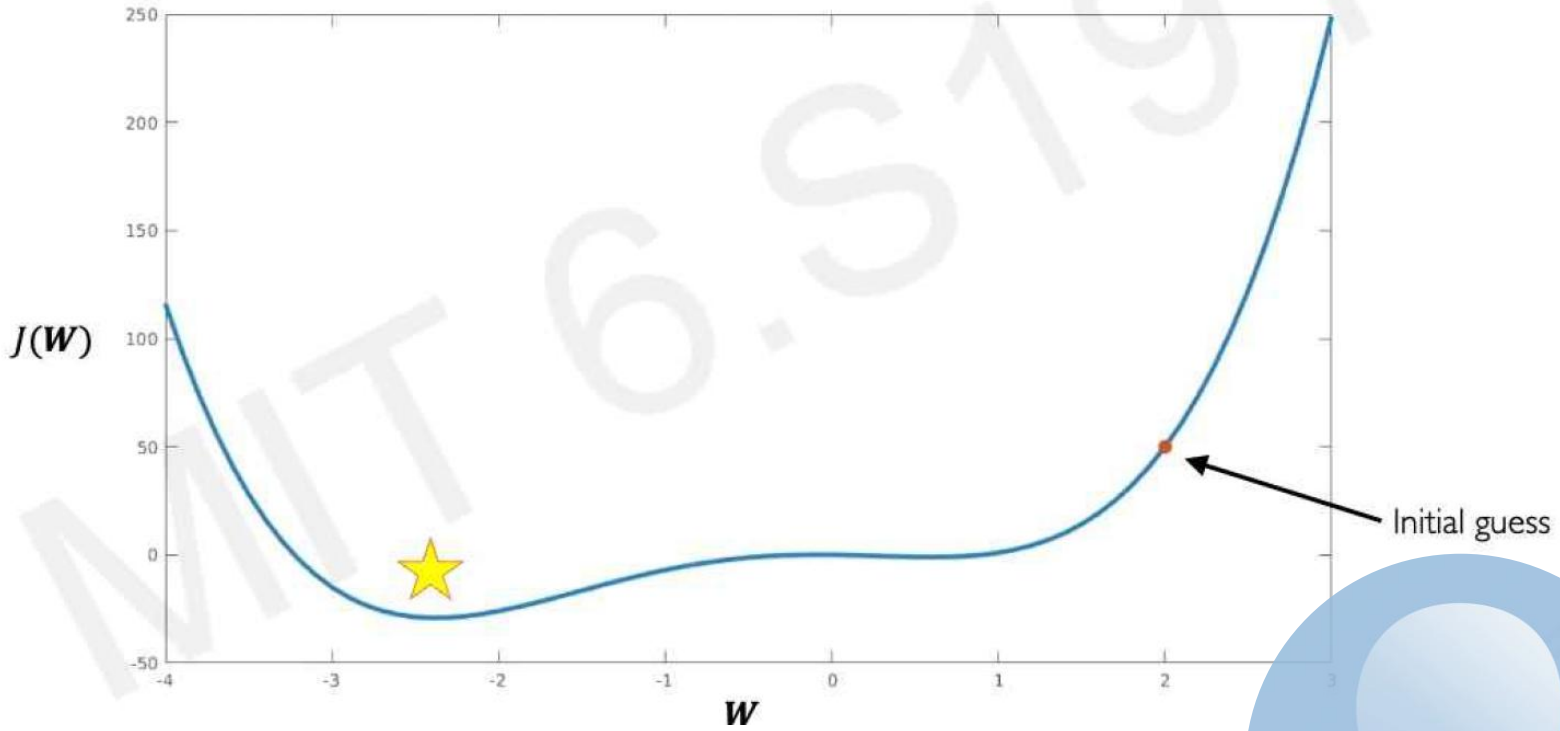
$$\boldsymbol{W} \leftarrow \boldsymbol{W} - \boxed{\eta} \frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}}$$

How can we set the learning rate?

MIT 6.S191

Setting the Learning Rate

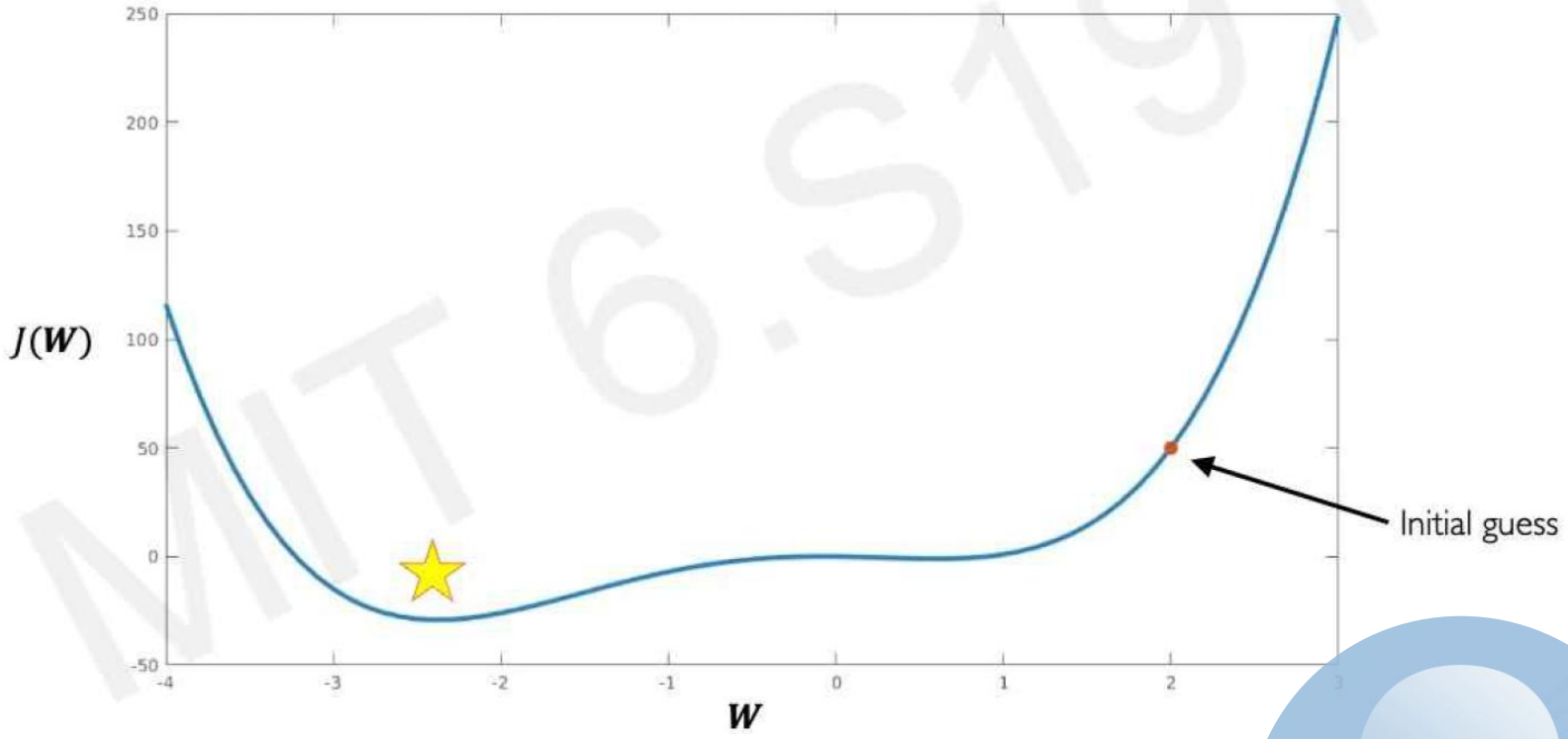
Small learning rate *converges slowly and gets stuck in false local minima*



MIT 6.S191

Setting the Learning Rate

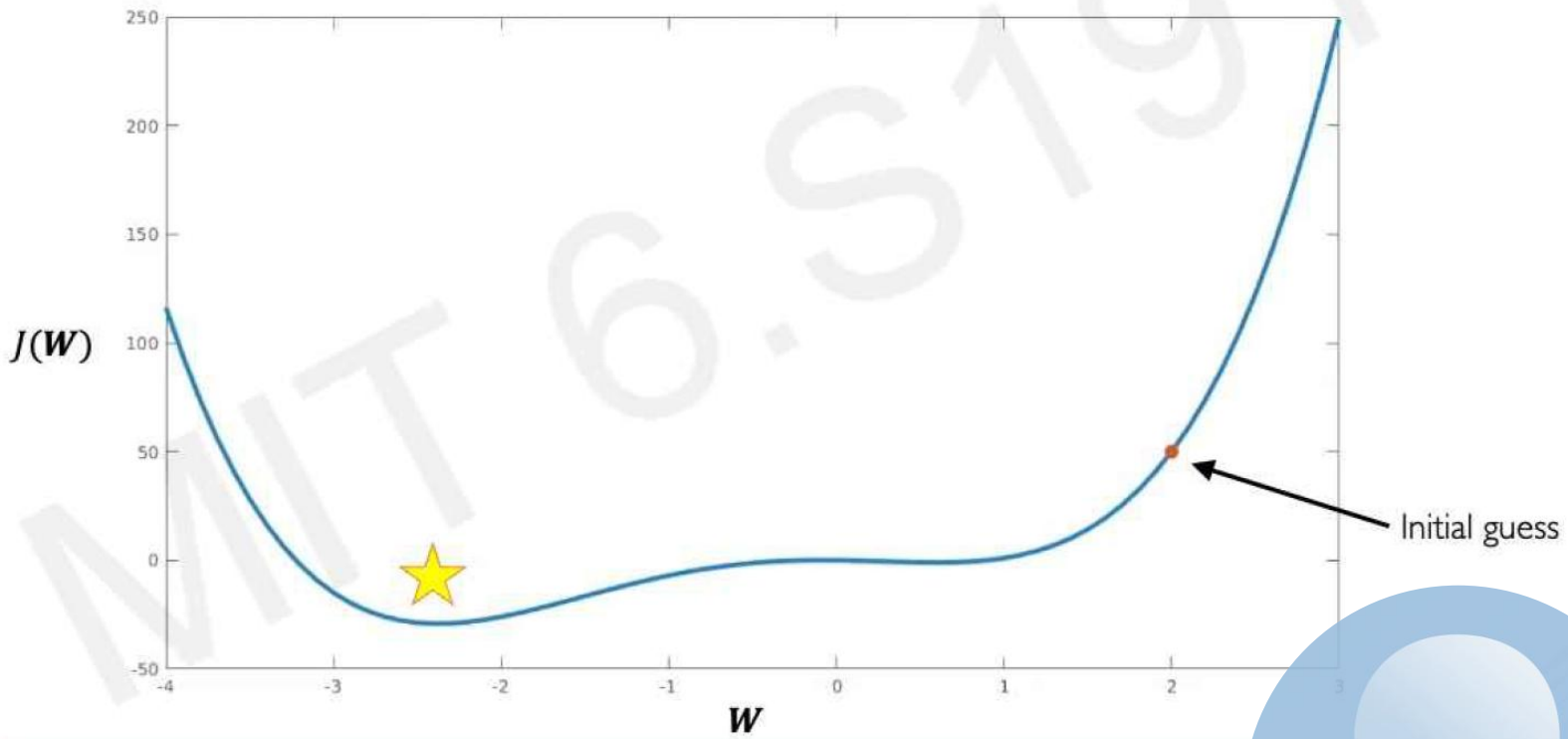
Large learning rates *overshoot, become unstable and diverge*



MIT 6.S191

Setting the Learning Rate

Stable learning rates *converge smoothly and avoid local minima*



MIT 6.S191

How to deal with this?

Idea 1:

Try lots of different learning rates and see what works "just right"

MIT 6.S191

How to deal with this?

Idea 1:

Try lots of different learning rates and see what works "just right"

Idea 2:

Do something smarter!
Design an adaptive learning rate that "adapts" to the landscape

MIT 6.S191

Adaptive Learning Rates

- Learning rates are no longer fixed
- Can be made larger or smaller depending on:
  - how large gradient is
  - how fast learning is happening
  - size of particular weights
  - etc...

MIT 6.S191

Gradient Descent Algorithms

Algorithm	TF Implementation	Torch Implementation	Reference
• SGD	tf.keras.optimizers.SGD	torch.optim.SGD	Kiefer & Wolfowitz, 1952.
• Adam	tf.keras.optimizers.Adam	torch.optim.Adam	Kingma et al., 2014.
• Adadelta	tf.keras.optimizers.Adadelta	torch.optim.Adadelta	Zeiler et al., 2012.
• Adagrad	tf.keras.optimizers.Adagrad	torch.optim.Adagrad	Duchi et al., 2011.
• RMSProp	tf.keras.optimizers.RMSProp	torch.optim.RMSProp	

Additional details: http://ruder.io/optimizing-gradient-descent/

Putting it all together

```
import tensorflow as tf

model = tf.keras.Sequential([...])

# pick your favorite optimizer
optimizer = tf.keras.optimizer.SGD()

while True: # loop forever

    # forward pass through the network
    prediction = model(x)

    with tf.GradientTape() as tape:
        # compute the loss
        loss = compute_loss(y, prediction)

    # update the weights using the gradient
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables)))
```

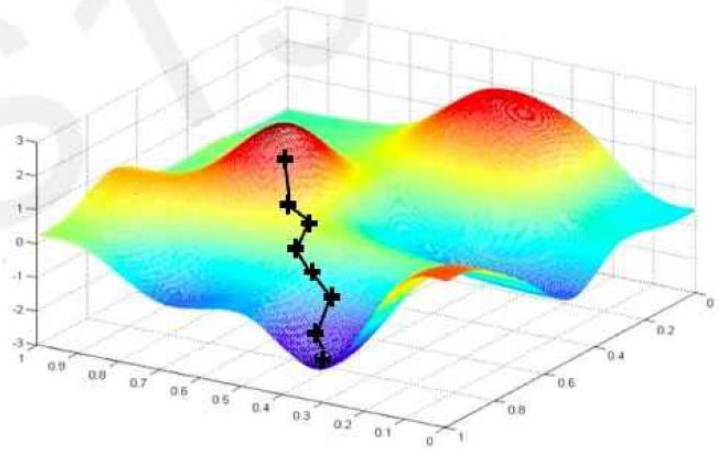
MIT 6.S191

Neural Networks in Practice: Mini Batches

Gradient Descent

Algorithm

1. Initialize weights randomly $\sim\mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}}$
4. Update weights, $\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta \frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}}$
5. Return weights

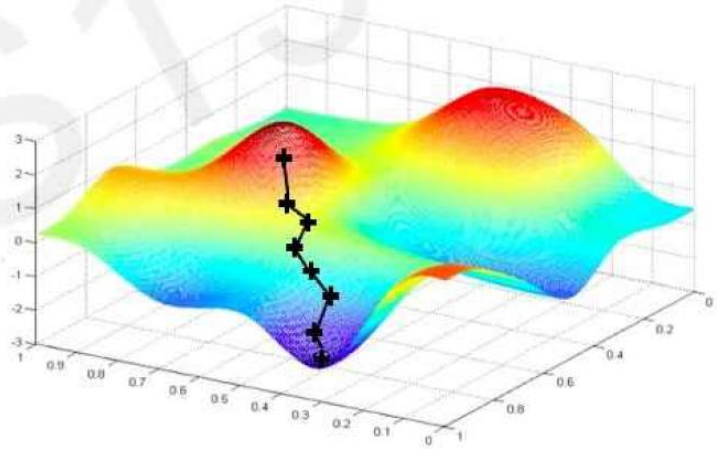


MIT 6.S191

Gradient Descent

Algorithm

1. Initialize weights randomly $\sim\mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}}$
4. Update weights, $\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta \frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}}$
5. Return weights



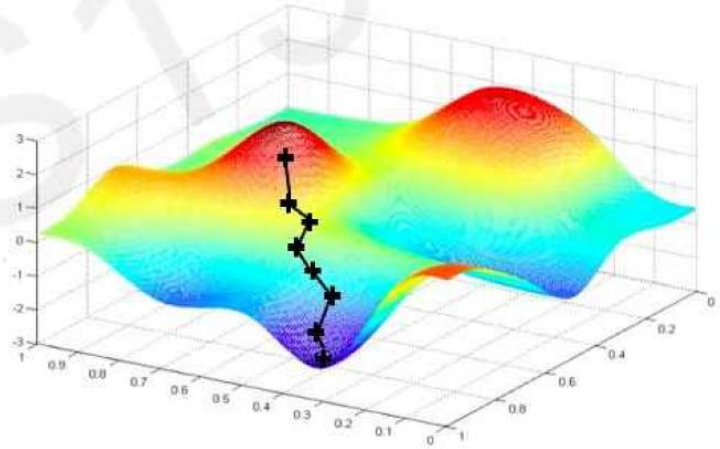
Can be very **computationally intensive** to compute!

MIT6.S191

Stochastic Gradient Descent

Algorithm

1. Initialize weights randomly $\sim\mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick single data point $\boldsymbol{i}$
4. Compute gradient, $\frac{\partial J_i(\boldsymbol{W})}{\partial \boldsymbol{W}}$
5. Update weights, $\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta \frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}}$
6. Return weights

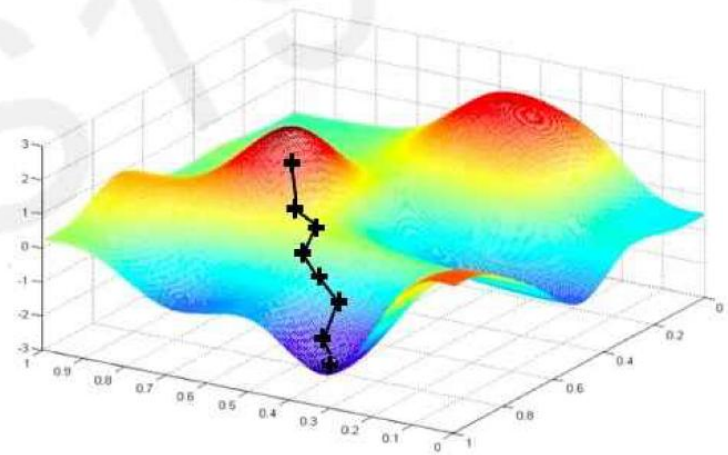


MIT 6.S191

Stochastic Gradient Descent

Algorithm

1. Initialize weights randomly $\sim\mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick single data point $\boldsymbol{i}$
4. Compute gradient, $\frac{\partial J_i(\boldsymbol{W})}{\partial \boldsymbol{W}}$
5. Update weights, $\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta \frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}}$
6. Return weights



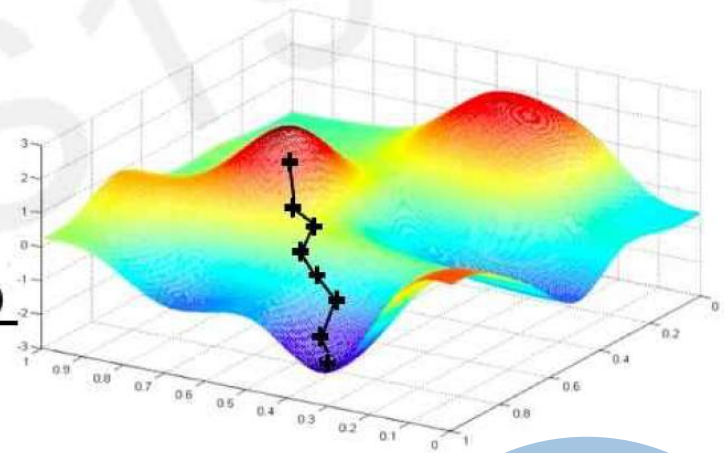
Easy to compute but **very noisy** (stochastic)!

MIT 6.S191

Stochastic Gradient Descent

Algorithm

1. Initialize weights randomly $\sim\mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick batch of B data points
4. Compute gradient, $\frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}} = \frac{1}{B}\sum_{k=1}^{B} \frac{\partial J_k(\boldsymbol{W})}{\partial \boldsymbol{W}}$
5. Update weights, $\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta \frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}}$
6. Return weights

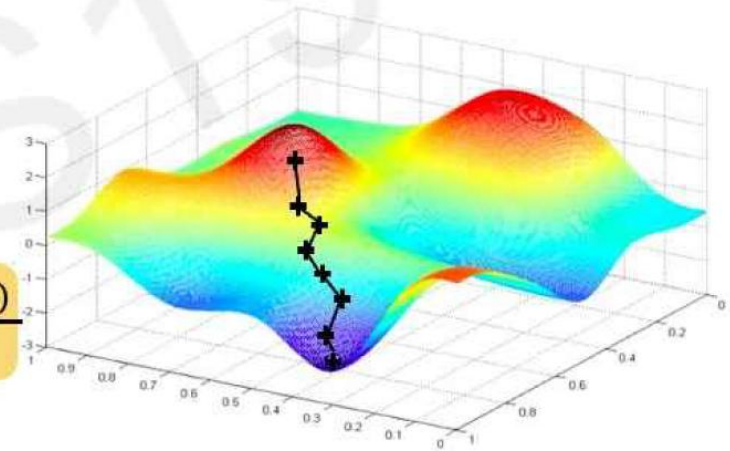


MIT 6.S191

Stochastic Gradient Descent

Algorithm

1. Initialize weights randomly $\sim\mathcal{N}(0,\sigma^2)$
2. Loop until convergence:
3. Pick batch of B data points
4. Compute gradient, $\boxed{\frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}} = \frac{1}{B}\sum_{k=1}^{B}\frac{\partial J_k(\boldsymbol{W})}{\partial \boldsymbol{W}}}$
5. Update weights, $\boldsymbol{W} \leftarrow \boldsymbol{W} - \eta\frac{\partial J(\boldsymbol{W})}{\partial \boldsymbol{W}}$
6. Return weights



Fast to compute and a much better estimate of the true gradient!

MIT 6.S191

Mini-batches while training

More accurate estimation of gradient

Smoother convergence

Allows for larger learning rates

MIT 6.S191

Mini-batches while training

More accurate estimation of gradient

Smoother convergence

Allows for larger learning rates

Mini-batches lead to fast training!

Can parallelize computation + achieve significant speed increases on GPU's

MIT 6.S191

Neural Networks in Practice: Overfitting

The Problem of Overfitting

Underfitting
Model does not have capacity to fully learn the data

Ideal fit

Overfitting
Too complex, extra parameters, does not generalize well

High bias

High variance

MIT 6.S191

Regularization

What is it?

Technique that constrains our optimization problem to discourage complex models

MIT 6.S191

Regularization

What is it?

Technique that constrains our optimization problem to discourage complex models

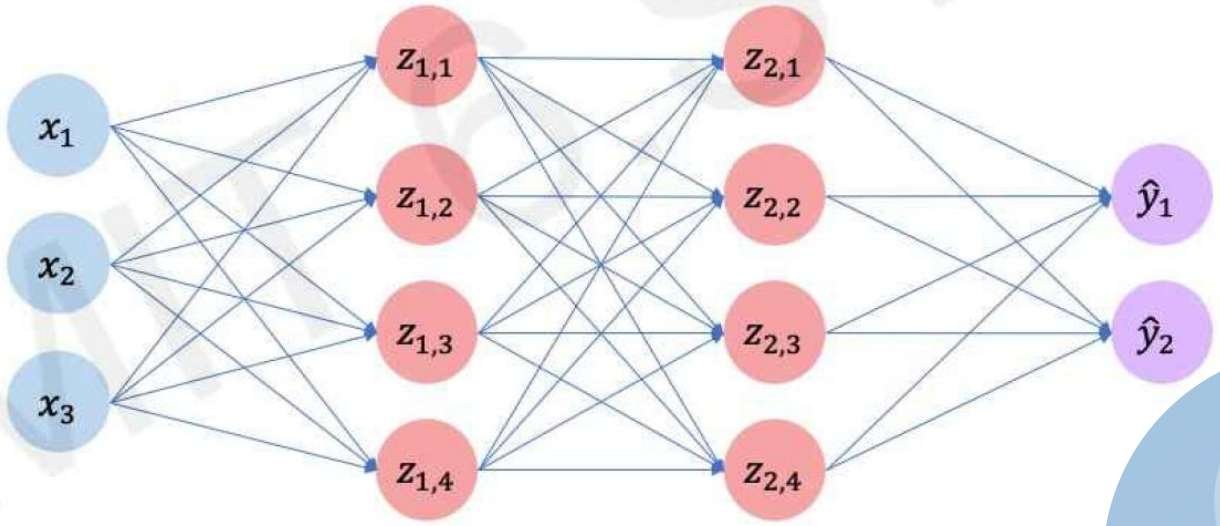
Why do we need it?

Improve generalization of our model on unseen data

MIT 6.S191

Regularization 1: Dropout

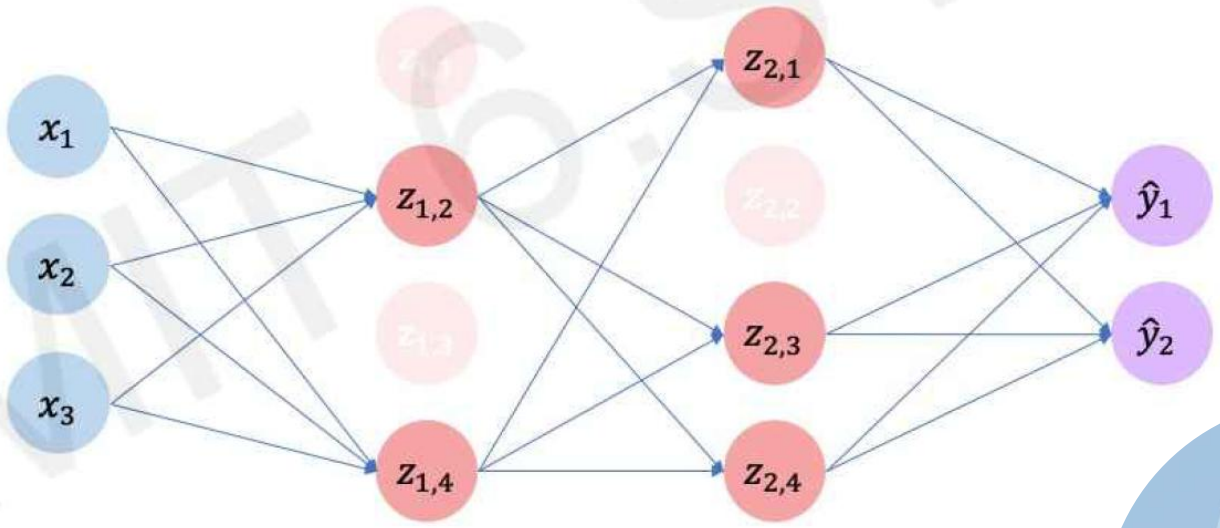
- During training, randomly set some activations to 0



Regularization 1: Dropout

- During training, randomly set some activations to 0
  - Typically 'drop' 50% of activations in layer
  - Forces network to not rely on any 1 node

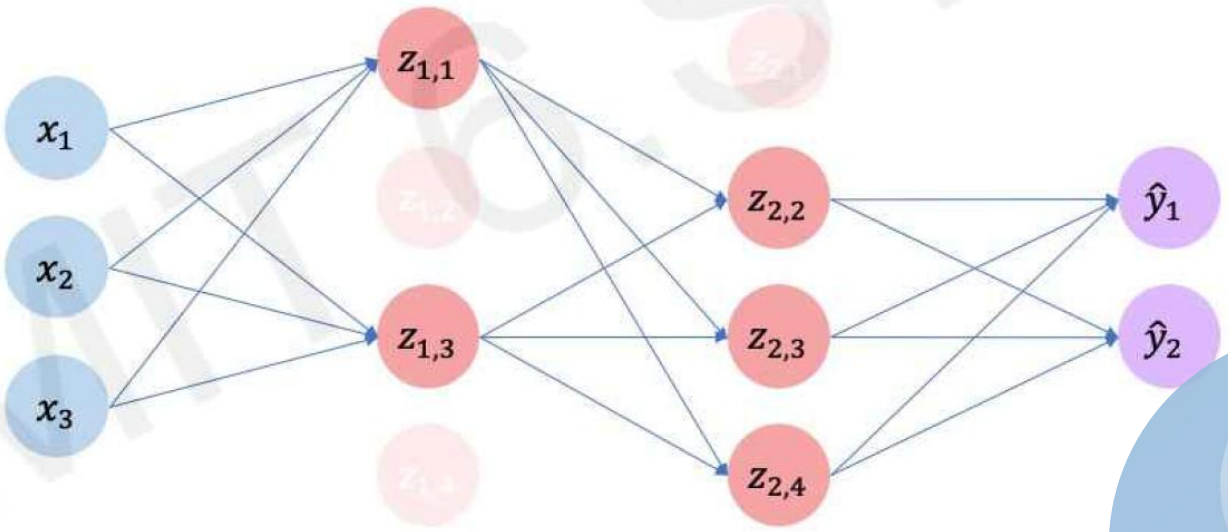
```
tf.keras.layers.Dropout(p=0.5)
torch.nn.Dropout(p=0.5)
```



Regularization 1: Dropout

- During training, randomly set some activations to 0
  - Typically 'drop' 50% of activations in layer
  - Forces network to not rely on any 1 node

```
tf.keras.layers.Dropout(p=0.5)
torch.nn.Dropout(p=0.5)
```



Regularization 2: Early Stopping

- Stop training before we have a chance to overfit

MIT 6.S191

Regularization 2: Early Stopping

- Stop training before we have a chance to overfit

MIT 6.S191

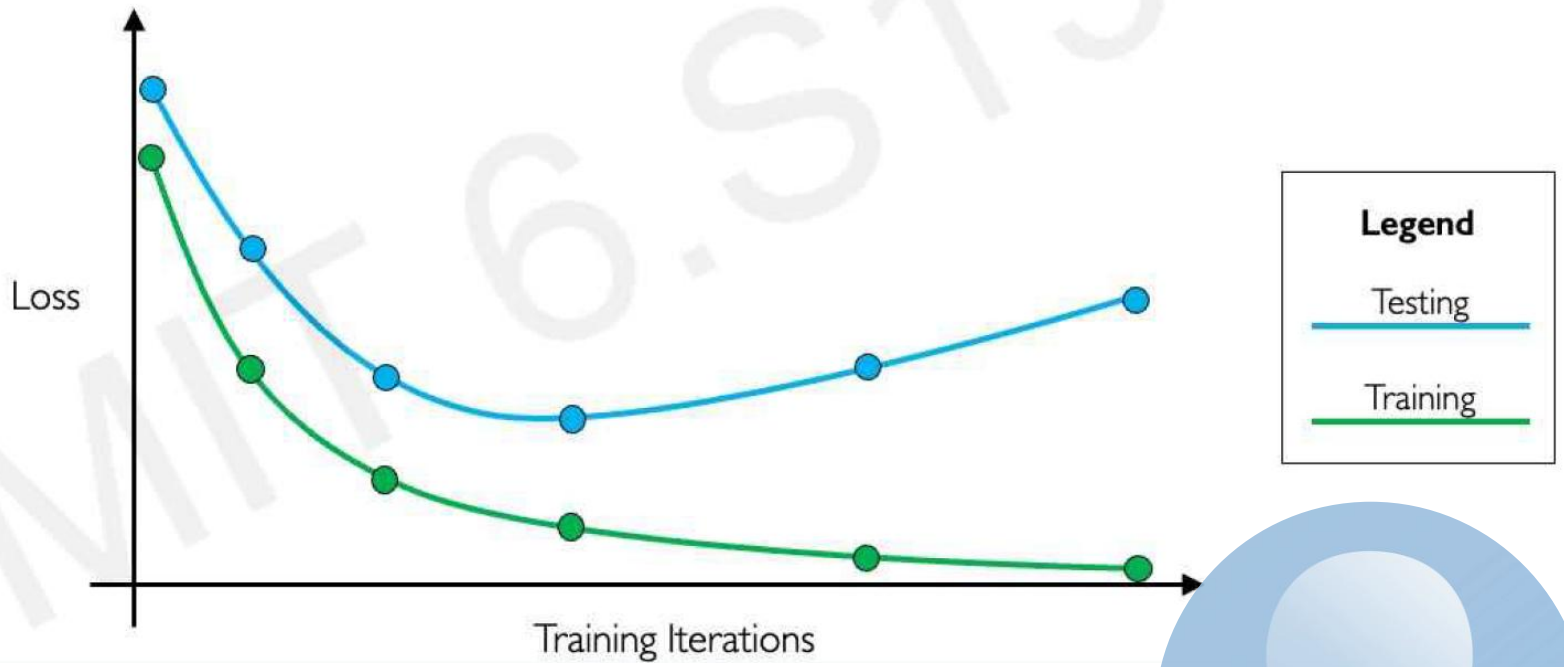
Regularization 2: Early Stopping

- Stop training before we have a chance to overfit

MIT 6.S191

Regularization 2: Early Stopping

- Stop training before we have a chance to overfit



MIT 6.S191

Regularization 2: Early Stopping

- Stop training before we have a chance to overfit

MIT 6.S191

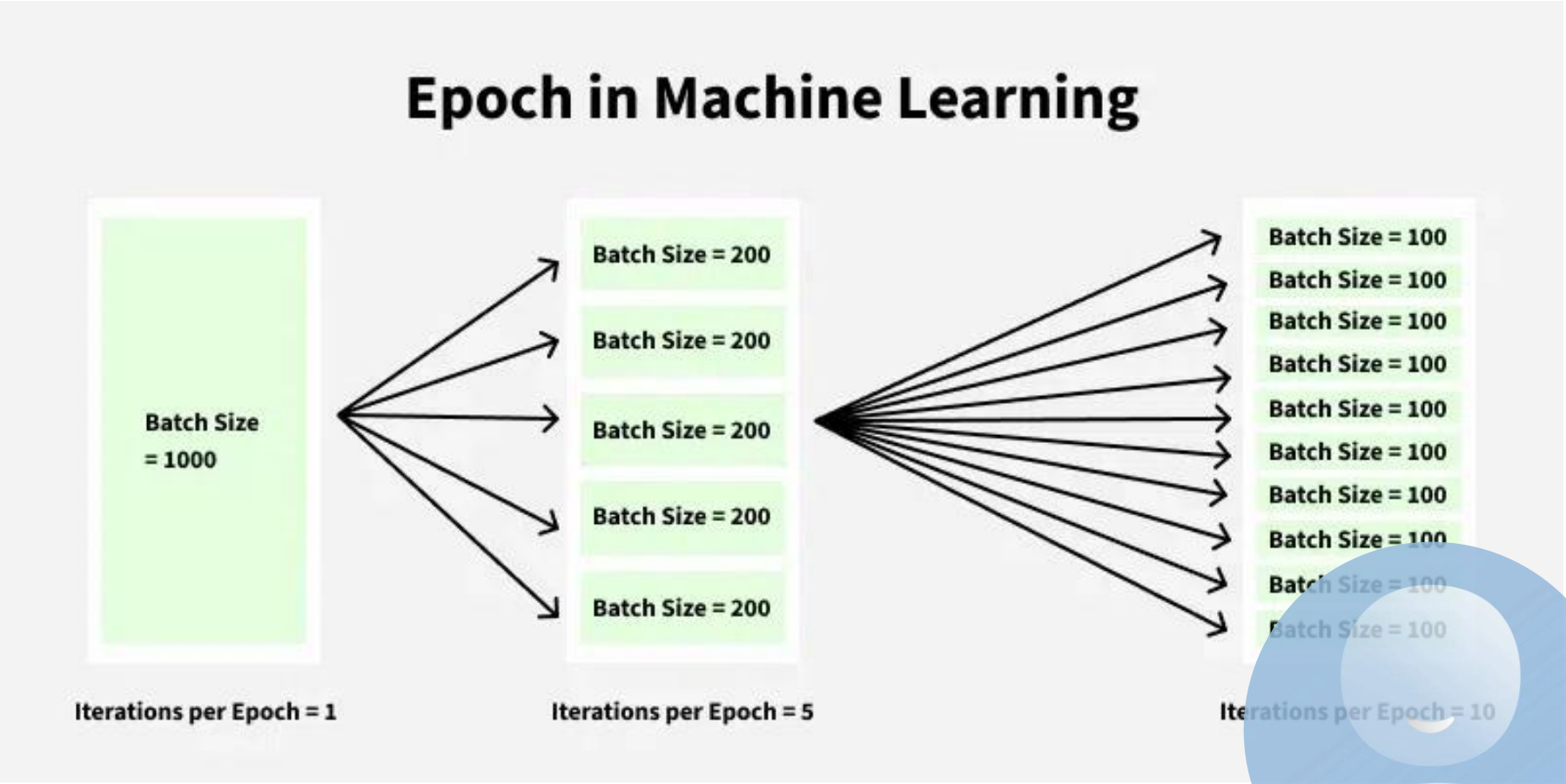
Regularization 2: Early Stopping

- Stop training before we have a chance to overfit

MIT 6.S191

Epoch in Machine Learning

One complete pass through the entire training dataset



How Epochs, Batches and Iterations Work Together?

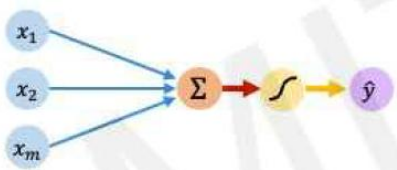
The relationship between ***epochs, batch size and iterations*** is important to optimize model training.

- **Epochs Ensure Data Completeness:** An epoch represents one complete pass through the entire training dataset, allowing the model to refine its parameters with each iteration.
- **Batch Size affects training efficiency:** The batch size refers to how many samples are processed in each batch. A larger batch size allows the model to process more data at once, smaller batches on the other hand provide more frequent updates.
- **Iterations update the model:** An iteration occurs each time a batch is processed where the model find the loss, adjusts its parameters and updates its weights based on that loss.

Core Foundation Review

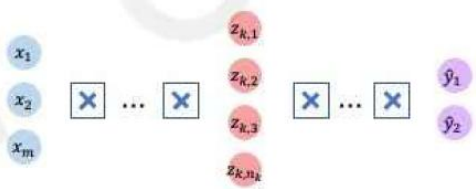
The Perceptron

- Structural building blocks
- Nonlinear activation functions



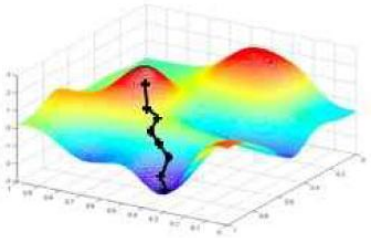
Neural Networks

- Stacking Perceptrons to form neural networks
- Optimization through backpropagation



Training in Practice

- Adaptive learning
- Batching
- Regularization



MIT 6.S191